Clustered Smart Home Alarm System Implementation (Pekko Cluster/Scala)

1. Problem Analysis

Transitioning the smart home alarm system to a distributed network via Apache Pekko Cluster alters location transparency, actor discovery and lifecycle management. The system must maintain identical state machine logic while operating across a dynamic network topology where keypad, sensors, and the control unit reside on separate JVMs.

Additional challenges include the fact that:

- Peripherals cannot rely on local actor paths or constructor-injected references. They must dynamically discover the active control unit across the cluster network.
- Inter-node communication protocol messages must cross network boundaries.
- A control unit crash/restart results in complete state loss. In such a case it should restart in a `SafeRecovery` state that can be escaped from only by inputting the correct PIN.

2. General Strategy and Implementation Details

Protocol Adaptation

The core protocol was updated to extend a `CborSerializable` marker trait, allowing all inter-node cluster messages to be serialized using **Jackson CBOR**. Additionally, a `ForceRestart` message was added to the `AlarmControlUnit` input protocol to trigger the recovery mechanism on demand.

Cluster Topology

The monolithic guardian has been redesigned. The new `AlarmSystemGuardian` is capable of spawning independent actor nodes based on cluster configuration roles, allowing the deployment of the control unit, keypad and sensors onto separate nodes rather than a single monolithic system.

- As an intentional implementation choice, the `AlarmControlUnitActor` and `SirenActor` are located on the same cluster node. Because of this, the `SirenActor` logic remains completely unchanged from the single-node implementation.
- Peripheral nodes like keypad and sensors run on separate cluster nodes and leverage the Pekko Cluster **Receptionist** subscription system to locate the `AlarmControlUnit`.

Peripheral Node States

To handle network latency and control unit restarts, both the `KeypadActor` and `SensorActor` implement a dual-behavior state machine:

- **Standby Behavior:** This is the initial state when the peripheral lacks a reference to the central control unit. The actor subscribes to the cluster `Receptionist` and listens for its `Listing` protocol updates. While in standby, the peripherals can still perform local actions such as buffering typed digits on the keypad or executing local intrusion detection, but they cannot interact with the control unit.
- **Active Behavior:** Triggered automatically once the `Receptionist` delivers a valid `Listing` containing the `AlarmControlUnitActor` reference. Upon transitioning to active, the peripheral can perform normal cluster interactions.

Control Unit Lifecycle

Upon creation, the `AlarmControlUnitActor` immediately registers itself with the cluster `Receptionist` by providing its designated `ServiceKey`.

To thoroughly test the recovery architecture, the control unit actor constructor includes a `simulateCrash` configuration option. If enabled, the actor sends itself a `ForceRestart` message once a given time mark has been reached. This message is handled in such a way that it intentionally forces the actor to crash, prompting the guardian's supervisor strategy to restart it.

Upon a supervisor-driven restart, the actor loses its current state and boots directly into the newly added `SafeRecovery` state:

- In this state, the control unit is powerless and cannot assume the home is safely armed or disarmed.
- It must receive a valid `PinEntered` message containing the correct PIN code to clear the recovery mode and return to the `Disarmed` state.

3. Demo

A complete demonstration of this distributed alarm system is provided via a **Docker Compose** setup; simply run `docker compose up --build` in the root folder to quickly spin up the entire cluster environment.

The test showcases identical functional state transitions to the monolithic version, but executes them in a distributed scenario. Additionally, the demo demonstrates Pekko Cluster node discovery and the lifecycle recovery procedure following a forced control unit restart.

Distributed Tic-Tac-Toe Implementation (Java RMI)

1. Problem Analysis

Implementing Tic-Tac-Toe as a distributed system using Java RMI introduces several challenges regarding state synchronization, remote reference management and concurrent programming. The primary one is the shift from implementing simple game logic to coordinating multiple clients that interact with a remote game state across separate JVMs.

From a concurrent and distributed point of view, the following critical aspects must be handled:

- A player creating a game or waiting for an opponent's turn must not freeze the application. The system must track remote game updates in a way that keeps the local user interface responsive while waiting for the other player to act.
- Multiple clients may attempt to create, join and view games simultaneously. Since RMI handles concurrent remote requests using a pool of server threads, operations such as creating a game instance, allocating a player slot and mutating the board grid must be synchronized to prevent race conditions.

2. General Strategy and Implementation Details

Architecture and Session Lifecycle

The system separates global session management from active gameplay using two distinct layers of remote interfaces:

- **GameManager**: Bound to the RMI registry under the identifier `"GameManager"`. It acts as the global entry point for clients, managing a collection of active game instances within a `ConcurrentHashMap`.
- **Game**: Instantiated dynamically upon a `createGame` request. When a client calls `joinGame`, the manager returns a stub of the specific `GameImpl` instance. Clients then interact directly with their designated game session.

Concurrency and Synchronization

Java RMI dispatches each incoming client request on a separate thread drawn from its internal pool, so shared state must be explicitly protected.

- `GameManagerImpl` synchronizes `createGame` and `joinGame` to ensure mutual exclusion during the game instance setup, preventing two or more clients from racing to create or join the same room simultaneously.
- `GameImpl` protects all state-mutating methods (`addPlayer`, `makeMove`,

`getBoard`, etc.) with `synchronized`, ensuring that turn verification, symbol placement, and win/draw evaluation are performed atomically.

Data Transfer and Remote References

- The `Board` state and `GameStatus` enum are serialized and transmitted by value when queried by clients.
- Both the main instance manager (`GameManagerImpl`) and the individual game instances (`GameImpl`) are remote objects managed by reference. When clients connect, they interact exclusively through remote proxy stubs, ensuring all actions are processed on the server side.

Client Design

Because remote calls in Java RMI are blocking operations, executing them directly on the user interface thread would cause the application to freeze. To prevent this, the client-side design isolates all network I/O from the Event Dispatch Thread (EDT):

- Since methods in `GameSession` like `createGame`, `joinGame`, `isMyTurn` and `makeMove` are blocking network calls, they are have been encapsulated inside a custom `runAsync` worker thread utility in the GUI. This guarantees that the user interface remains responsive while waiting for the server operations to complete.
- Game state synchronization is handled by a `ScheduledExecutorService` inside `GameSession`. This background scheduler continuously pulls the current board status from the server every 500ms without blocking the GUI.

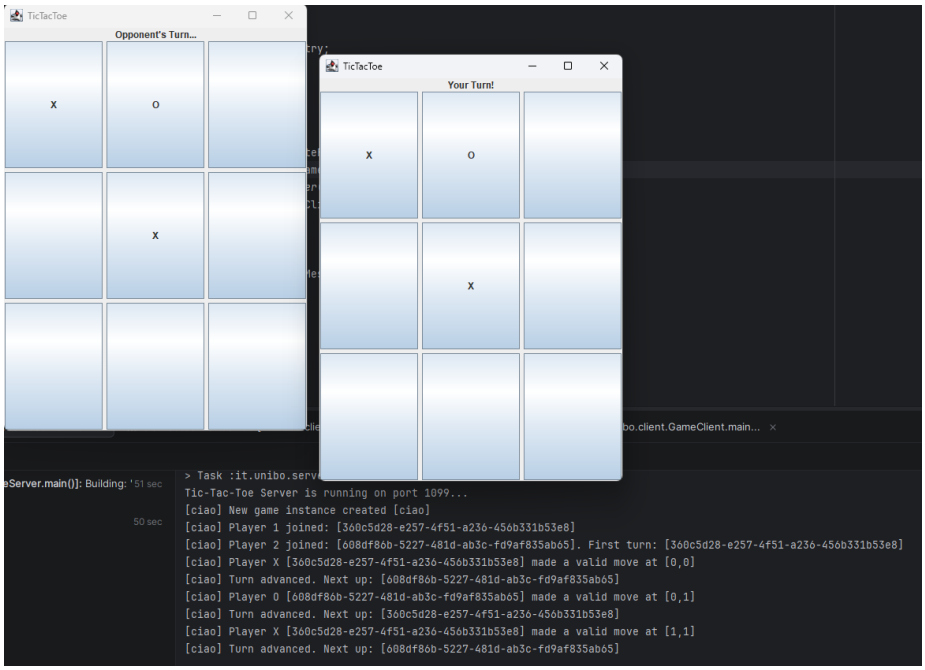

Figure 1: Two clients connected to the same game session, with server logs showing move history and turn advancement